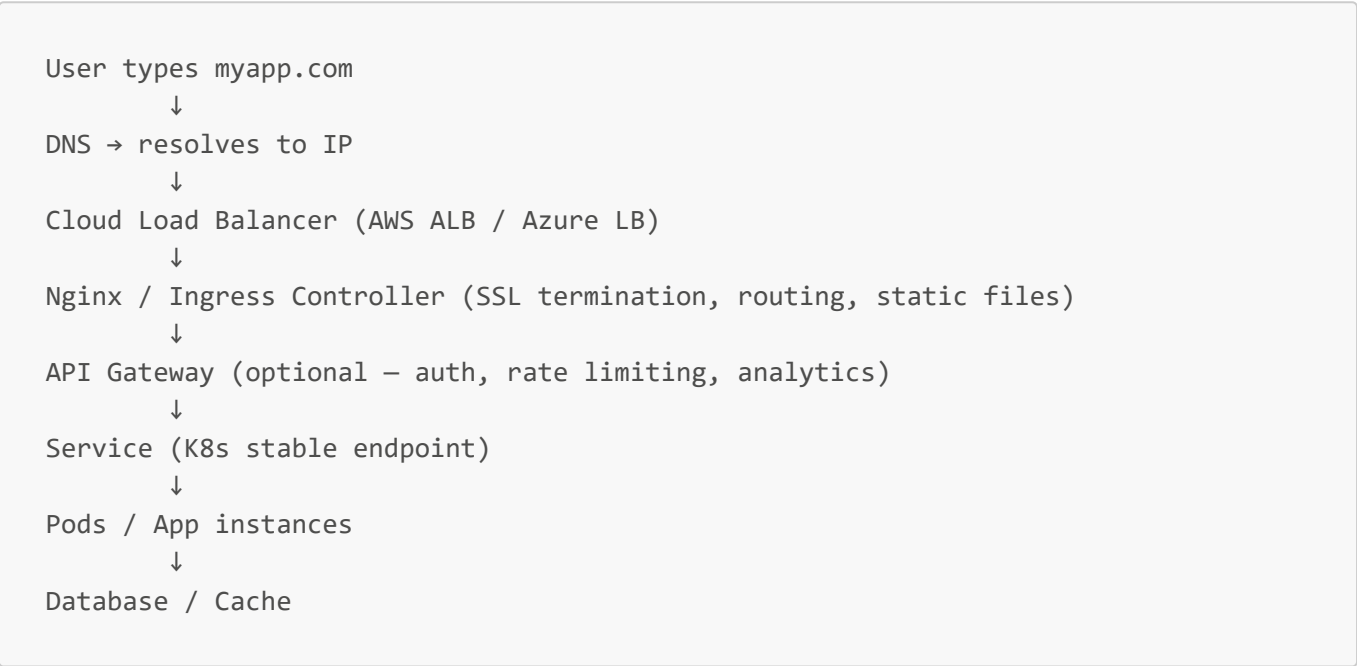


14 — Web Servers

Key Concepts

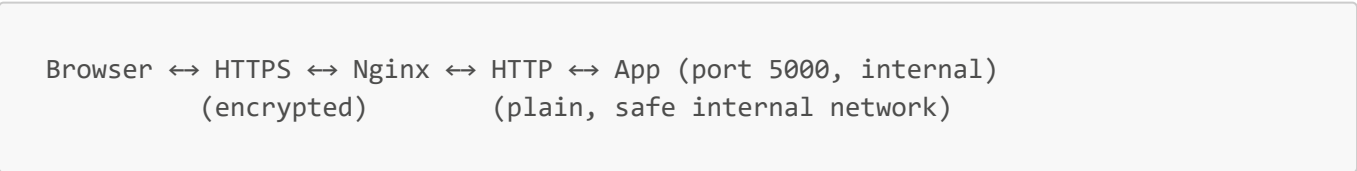
- **Web server** = Software that receives HTTP requests and sends responses
 - Sits in front of your app — app is never exposed directly to the internet
 - Handles: static files, SSL termination, reverse proxying, load balancing, rate limiting
 - Main options: Nginx, Apache, Caddy, MS IIS
-

Full Request Flow



Reverse Proxy

Nginx sits between internet and your app:



Why app is never exposed directly:

- No SSL handling built in
- No rate limiting or load balancing
- Security risk — only Nginx faces the internet

What Nginx does:

Responsibility	Detail
----------------	--------

Responsibility	Detail
SSL termination	Decrypts HTTPS, forwards plain HTTP to app
Static file serving	Serves HTML/CSS/JS/images directly — app not involved
Reverse proxying	Forwards API requests to app server
Load balancing	Distributes across multiple app instances
Rate limiting	Blocks too many requests from one IP
Caching	Caches responses to reduce app load

Static vs Dynamic Request Handling

```
GET /images/logo.png → Nginx serves file directly from disk (no app)
GET /api/search?q=x → Nginx proxies to Node.js/NET app → returns JSON
```

Nginx vs Apache

	Nginx	Apache
Architecture	Event-driven, non-blocking	Thread/process per connection
Concurrency	Handles more with less memory	More memory under high load
Config	Centralized config files	Per-directory <code>.htaccess</code>
Static files	Faster	Good
Dynamic content	Proxies to app server	Can handle directly (mod_php)
Best for	High-traffic, reverse proxy, K8s	Shared hosting, flexibility

Caddy

- Modern web server with **automatic HTTPS** via Let's Encrypt
- Zero manual SSL certificate management
- Minimal config:

```
myapp.com {
  reverse_proxy localhost:5000
}
```

- Best for: small teams, side projects, developer tools

MS IIS

- Microsoft's web server — built into Windows Server
- Standard for hosting .NET Framework / ASP.NET on Windows
- Deep Windows integration: Active Directory, Windows Auth
- Modern .NET Core: IIS acts as reverse proxy in front of **Kestrel**

Internet → IIS (reverse proxy) → Kestrel (ASP.NET Core app)

	IIS	Nginx
Platform	Windows only	Cross-platform
Best for	.NET Framework, Windows Auth	High-traffic, Linux
Management	GUI (IIS Manager)	Config files

Nginx vs API Gateway

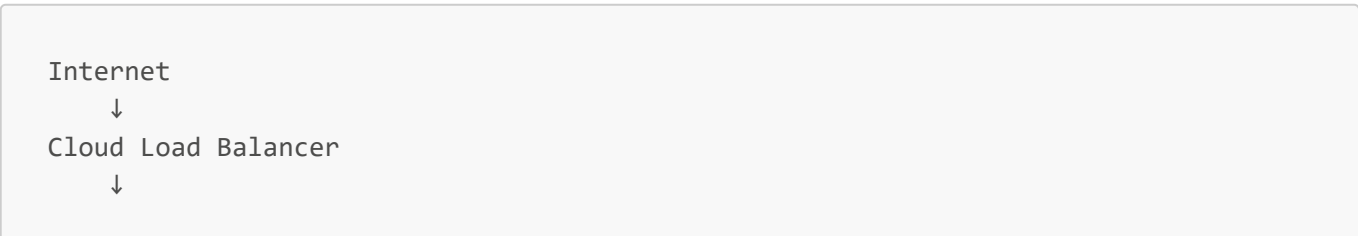
	Nginx	API Gateway (Kong, AWS API GW)
Level	Infrastructure	Application
SSL	Yes	Yes
Load balancing	Yes	Yes
JWT/OAuth auth	No	Yes — centralized
Rate limiting	Basic	Advanced (per user, per API key)
Analytics	No	Yes
Request transformation	No	Yes

Production pattern — both used together:

Browser → Nginx (SSL, static files) → API Gateway (auth, routing) → Microservices

Kubernetes — Ingress Controller

In K8s, Nginx runs **as a Pod** inside the cluster as the Ingress Controller:



```
Ingress Controller (Nginx Pod) – routing rules, SSL termination
↓
Service (stable ClusterIP)
↓
App Pods
```

Industry Examples

Server	Company / Use Case
Nginx	Netflix, Airbnb, GitHub — high-traffic reverse proxy
Apache	Wikipedia, traditional shared hosting
Caddy	Small startups, developer tools — automatic HTTPS
IIS	Enterprise .NET on Windows, government systems

Mental Model — Airport Analogy

Arrive at airport	= DNS resolves domain
Main entrance gate	= Cloud Load Balancer
Security checkpoint	= Nginx / Ingress Controller
Passport control	= API Gateway (auth, rate limiting)
Departure gate	= K8s Service
Your plane	= Pod (actual app)

Interview Questions

Q1: What is a web server?

Receives HTTP requests and sends responses — serves static files, proxies to app, handles SSL and load balancing.

Q2: What is a reverse proxy and why use one?

Sits between internet and app. Handles SSL termination, static files, load balancing, rate limiting. App is never exposed directly — security and capability reasons.

Q3: Static vs dynamic request — how does Nginx handle differently?

Static files served directly from disk — app not involved. Dynamic API requests proxied to app server.

Q4: What is SSL termination?

Browser talks HTTPS to Nginx. Nginx decrypts and forwards plain HTTP to app internally — app doesn't handle SSL.

Q5: Nginx vs Apache?

Nginx is event-driven — one worker handles thousands of connections. Apache creates thread per connection — more memory under load. Nginx wins on concurrency; Apache on flexibility.

Q6: Where does Nginx sit in Kubernetes?

As the Ingress Controller — Nginx runs as a Pod, handles routing and SSL termination, forwards to Services which route to Pods.

Q7: Nginx vs API Gateway?

Nginx = infrastructure proxy (SSL, load balancing). API Gateway = application layer (JWT auth, per-client rate limiting, analytics). Production uses both.